

Software Requirements Specification

for

Code Similarity Platform

Academic Integrity Submission Similarity Analysis System

Version: 2.0

Prepared by: Group 16 — Muhammad Nabeel Waheed, Thomas Neal, Ghassan Balouze,
Kev K Akpinar, Abdel Zahran, Ty Mabee

Kool Group Name: Last Minute Warriors

Course: COSC 4P02 — Course Project

Date: 2026-04-24

Contents

1	Introduction	3
1.1	Document Purpose	3
1.2	Document Conventions	3
1.3	Project Scope	3
1.4	References	3
1.5	Intended Use of GitHub	4
2	Overall Description	4
2.1	Product Perspective	4
2.2	System Context Summary	5
2.3	User Classes and Characteristics	5
2.4	Operating Environment	6
2.5	Design and Implementation Constraints	6
2.6	Assumptions and Dependencies	7
3	System Features	7
3.1	Professor Authentication and Protected Access	7
3.1.1	Description	7
3.1.2	Stimulus/Response Sequences	7
3.1.3	Functional Requirements	7
3.2	Assignment Management	7
3.2.1	Description	7
3.2.2	Stimulus/Response Sequences	8
3.2.3	Functional Requirements	8
3.3	Submission Upload and Status Tracking	8
3.3.1	Description	8
3.3.2	Stimulus/Response Sequences	8
3.3.3	Functional Requirements	8
3.4	Template Management and Mandatory Template Subtraction	10
3.4.1	Description	10
3.4.2	Stimulus/Response Sequences	10
3.4.3	Functional Requirements	10
3.5	Historical Submission Upload for Cross-Term Comparison	10
3.5.1	Description	10
3.5.2	Stimulus/Response Sequences	10
3.5.3	Functional Requirements	11
3.6	Similarity Analysis Jobs	11
3.6.1	Description	11
3.6.2	Stimulus/Response Sequences	11
3.6.3	Functional Requirements	12
3.7	Professor Review and Pair Comparison View	13
3.7.1	Description	13
3.7.2	Stimulus/Response Sequences	13
3.7.3	Functional Requirements	13
3.8	Privacy and Identity Protection	13
3.8.1	Description	13

3.8.2	Stimulus/Response Sequences	14
3.8.3	Functional Requirements	14
4	Data Requirements	14
4.1	Logical Data Model	14
4.2	Data Dictionary	14
4.3	Data Acquisition, Integrity, Retention, and Disposal	16
5	External Interface Requirements	16
5.1	User Interfaces	16
5.2	Software Interfaces	17
5.3	Hardware Interfaces	17
5.4	Communications Interfaces	17
6	Quality Attributes	18
6.1	Usability	18
6.2	Performance	18
6.3	Security	18
7	Use Cases	19
7.1	UC-01: Public Student Submits an Assignment	19
7.1.1	Main Success Scenario	19
7.1.2	Alternate / Exception Flows	19
7.2	UC-02: Professor Uploads Template or Historical Archive	19
7.2.1	Main Success Scenario	19
7.2.2	Alternate / Exception Flows	20
7.3	UC-03: Professor Runs Similarity Analysis	20
7.3.1	Main Success Scenario	20
7.3.2	Alternate / Exception Flows	21
7.4	UC-04: Professor Reviews Suspicious Pairs	22
7.4.1	Main Success Scenario	22
7.4.2	Alternate / Exception Flows	22
8	UML and Design Documentation	22
8.1	Key Architectural Decisions	22
8.2	Diagram Update Notes for Version 2.0	23
9	Glossary	23

1 Introduction

1.1 Document Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for the implemented Similarity App. Version 2.0 updates the earlier Phase 1 SRS so that the document matches the completed application now present in the repository. The intended audience includes instructors, developers, testers, maintainers, and deployers.

1.2 Document Conventions

Requirements are labeled using the following identifiers:

- **FR-**: Functional Requirement
- **NFR-**: Non-Functional Requirement
- **DR-**: Data Requirement
- **UC-**: Use Case

Shall indicates a mandatory requirement. **Should** indicates a strong recommendation.

1.3 Project Scope

The system provides a web-based platform where:

- students submit programming assignments through a public upload flow using an assignment key,
- TAs or markers may use a bulk-upload flow on the same public submission page,
- professors authenticate into a dashboard to create and manage assignments,
- professors upload historical archives and template code,
- professors manually trigger similarity analysis runs,
- and professors review suspicious pairs through a side-by-side evidence viewer.

The system compares current submissions against other submissions in the same assignment and against historical submissions uploaded for that assignment. Minimum language support is C, C++, and Java. The similarity engine is implemented within the project and is not delegated to an external AI service.

1.4 References

- Project Statement (provided by instructor)
- Phase 1 SRS Version 1.0
- Implemented monorepo repository and source code
- Prisma schema, migrations, and runtime documentation in the repository

1.5 Intended Use of GitHub

Project coordination, issue tracking, milestones, documentation, and version control shall be maintained in the shared GitHub repository. Implementation-aligned documentation updates, including this SRS, shall be versioned with the source code.

2 Overall Description

2.1 Product Perspective

The product is a web application composed of:

- a public-facing browser client for assignment-key based submission,
- a professor-facing browser client for assignment management and review,
- a backend API server,
- a PostgreSQL database for metadata,
- a Redis-backed queue for background jobs,
- a background worker for upload preparation and comparison orchestration,
- a Rust-based command-line engine for similarity analysis,
- and filesystem-backed object storage for uploaded and prepared artifacts.

The system is assignment-centric. The earlier course/offering abstraction is not part of the implemented application.

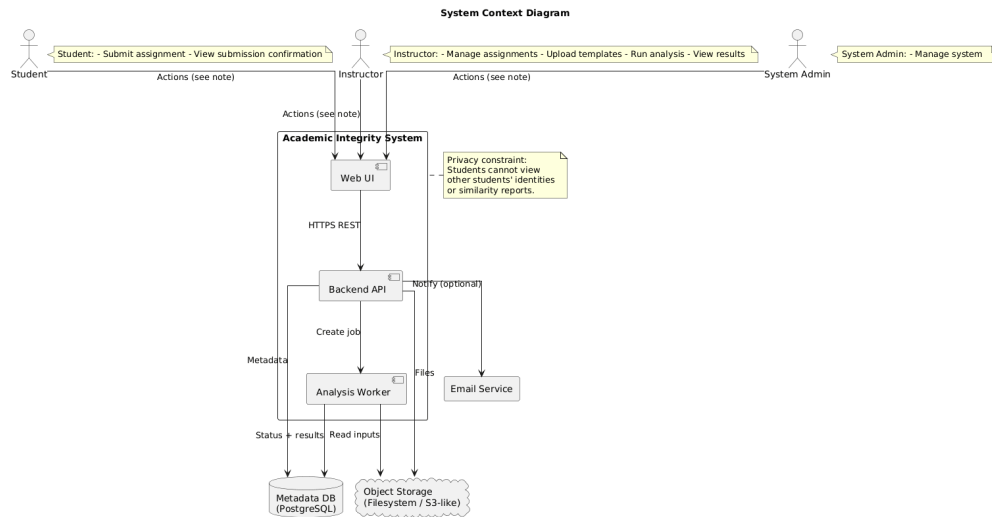


Figure 1: System Context Diagram

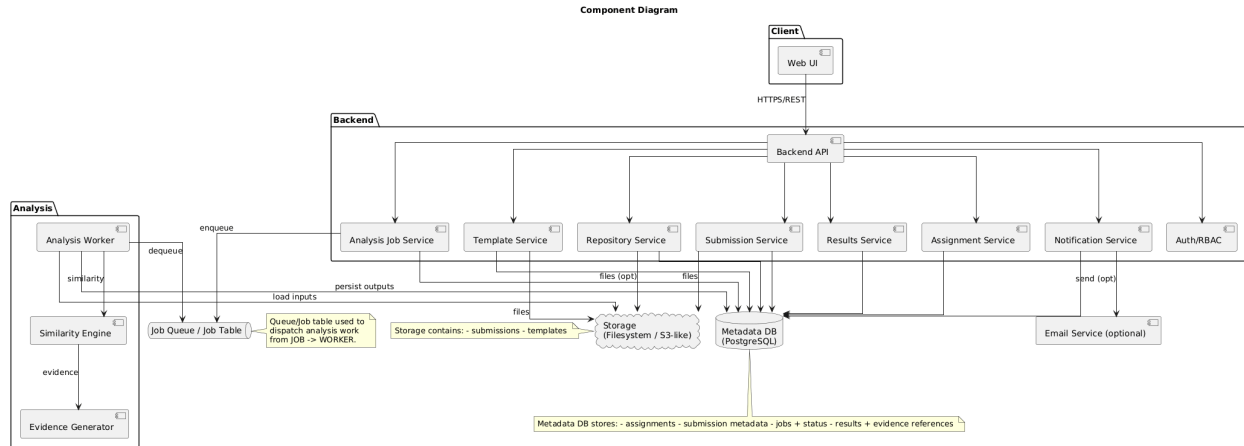


Figure 2: Component Diagram

2.2 System Context Summary

The implemented runtime flow is:

1. A student, TA, or marker submits a zip archive through the public web UI using an assignment key.
2. The API stores metadata, stages the archive, and queues a preparation job.
3. The worker extracts and prepares the archive into current, historical, or template artifacts.
4. A professor authenticates into the dashboard, manages assignments, and manually starts a comparison run.
5. The worker launches the Rust engine, passes a JSON analysis request over stdin, and receives JSON results over stdout.
6. Comparison results are stored in PostgreSQL and displayed in the professor UI.

2.3 User Classes and Characteristics

- **Student Submitter:** uses a public upload form, provides identity fields and an assignment key, uploads exactly one zip archive, and receives upload-status feedback for that submission.
- **TA / Marker Bulk Submitter:** uses the bulk-upload section on the public page, provides an assignment key, uploads one parent zip containing child submission zips, and receives upload-status feedback.
- **Professor:** authenticates through the professor login flow, creates and manages assignments, uploads historical archives and templates, triggers comparisons, reviews suspicious pairs, reveals identity when authorized, and downloads assignment artifacts.
- **System Administrator:** manages deployment, environment configuration, database migration, bootstrap account creation, and operational hosting. This is an operational role, not an in-application RBAC role.

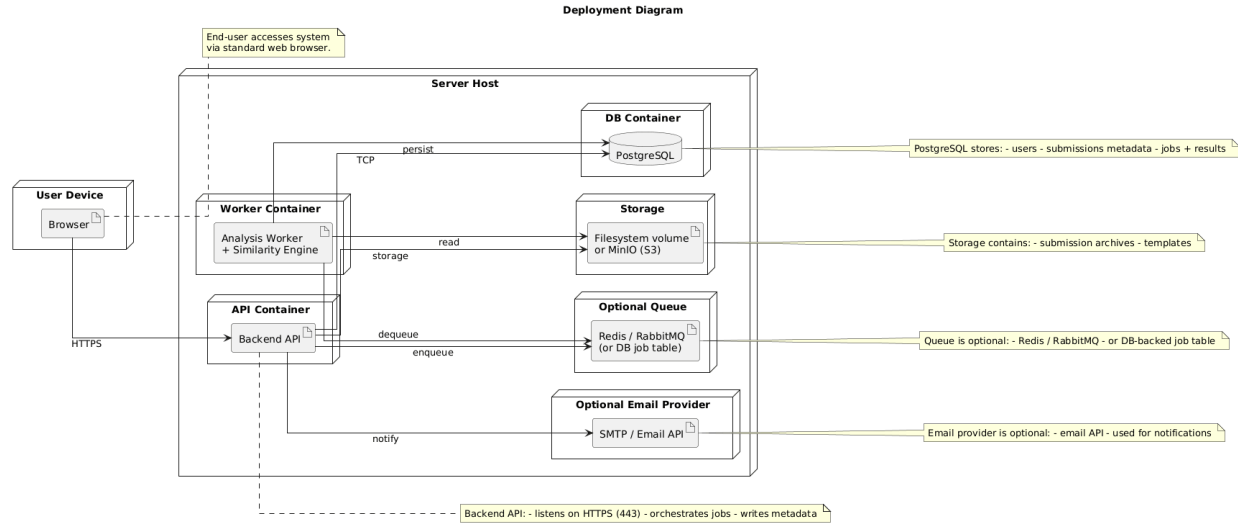


Figure 3: Deployment Diagram

2.4 Operating Environment

- **Client:** modern web browsers (Chrome, Firefox, Safari, Edge).
- **Server:** Linux-based host or container platform running the web app, API, worker, PostgreSQL, and Redis.
- **Storage:** PostgreSQL for metadata and filesystem-backed object storage for raw and prepared archives.
- **Network:** HTTPS for deployed browser traffic; internal service communication over local/container networking.

2.5 Design and Implementation Constraints

- **FR-001** The system shall support C, C++, and Java at minimum.
- **NFR-001** The code-comparison function shall not be outsourced to an external AI service.
- **NFR-002** The system shall protect submission identity information and restrict identity revelation to authorized professor workflows.
- **NFR-003** The system shall use HTTPS for client-server communications in non-local deployments.
- **NFR-004** The runtime deployment shall include PostgreSQL, Redis, shared artifact storage, and an engine binary available to the worker.
- **FR-002** Template subtraction shall be applied before final similarity scoring.

2.6 Assumptions and Dependencies

- Public submitters receive a valid assignment key from the professor or course staff.
- Professors manage assignments directly; the application does not model higher-level course/offering entities.
- Similarity accuracy depends on valid archive structure, supported source files, and correct language-specific parsing.
- Comparisons are started manually by a professor and are not triggered automatically on every upload.

3 System Features

3.1 Professor Authentication and Protected Access

3.1.1 Description

Priority: High. Supports login, signup, session handling, and authorization for professor-only functionality. Student-facing submission flows are public and key-based rather than account-based.

3.1.2 Stimulus/Response Sequences

- Professor signs up or logs in → system creates or validates a session → professor dashboard is displayed.
- Unauthenticated user requests a professor-only resource → system denies access.
- Professor requests a protected assignment or comparison resource → system verifies assignment ownership → resource is returned.

3.1.3 Functional Requirements

1. **FR-003** The system shall authenticate professors before allowing access to protected dashboard functionality.
2. **FR-004** The system shall support professor login and professor signup.
3. **FR-005** The system shall enforce authorization checks on protected API endpoints.
4. **FR-006** The system shall restrict assignment management, comparison runs, pair review, identity reveal, and artifact download to the professor who manages the assignment.
5. **FR-007** The system shall expose public submission endpoints separately from professor-only endpoints.

3.2 Assignment Management

3.2.1 Description

Priority: High. Allows professors to create and manage assignments, define assignment language and due date, obtain an assignment key, and monitor comparison readiness for each assignment.

3.2.2 Stimulus/Response Sequences

- Professor creates an assignment → system stores assignment metadata and generates an active assignment key.
- Professor edits a due date → system stores the new due date and refreshes dashboard details.
- Professor opens an assignment workspace → system shows submissions, templates, uploads, and comparison state.
- Professor finalizes uploads and starts the comparison engine → system validates readiness and queues a comparison run.

3.2.3 Functional Requirements

1. **FR-008** The system shall allow professors to create assignments.
2. **FR-009** The system shall store assignment metadata including title, language, due date, and generated assignment key.
3. **FR-010** The system shall allow professors to update an assignment due date.
4. **FR-011** The system shall display assignment-level readiness state for comparison processing.
5. **FR-012** The system shall allow professors to delete an assignment when no uploads or comparison runs are active.

3.3 Submission Upload and Status Tracking

3.3.1 Description

Priority: High. Supports public student submission by assignment key, optional TA/marker bulk current upload, durable archive storage, metadata tracking, and upload-status feedback.

3.3.2 Stimulus/Response Sequences

- Public submitter enters identity fields, assignment key, and zip archive → system validates input, encrypts identity in the browser, stores the upload, and returns upload status tracking data.
- Public submitter polls upload status → system returns upload-batch status, warnings, failure reason if any, and prepared submission names when ready.
- TA or marker uploads a bulk current archive → system extracts child zips and creates one current submission per child zip.

3.3.3 Functional Requirements

1. **FR-013** The system shall accept student submissions through a public web-based upload flow using an assignment key.
2. **FR-014** The system shall require uploads to be zip archives.

3. **FR-015** The system shall store raw uploaded archives in server-side storage and maintain upload metadata in the database.
4. **FR-016** The system shall return an upload batch identifier and status-tracking capability after a public upload is accepted.
5. **FR-017** The public UI shall allow a submitter to view whether that upload has been received, processed successfully, or failed.
6. **FR-018** The system shall support a bulk current-submission upload flow for TA or marker use, where one parent zip contains child submission zips.
7. **FR-019** The system shall retain current and historical submissions according to assignment rules and authorized deletion actions.

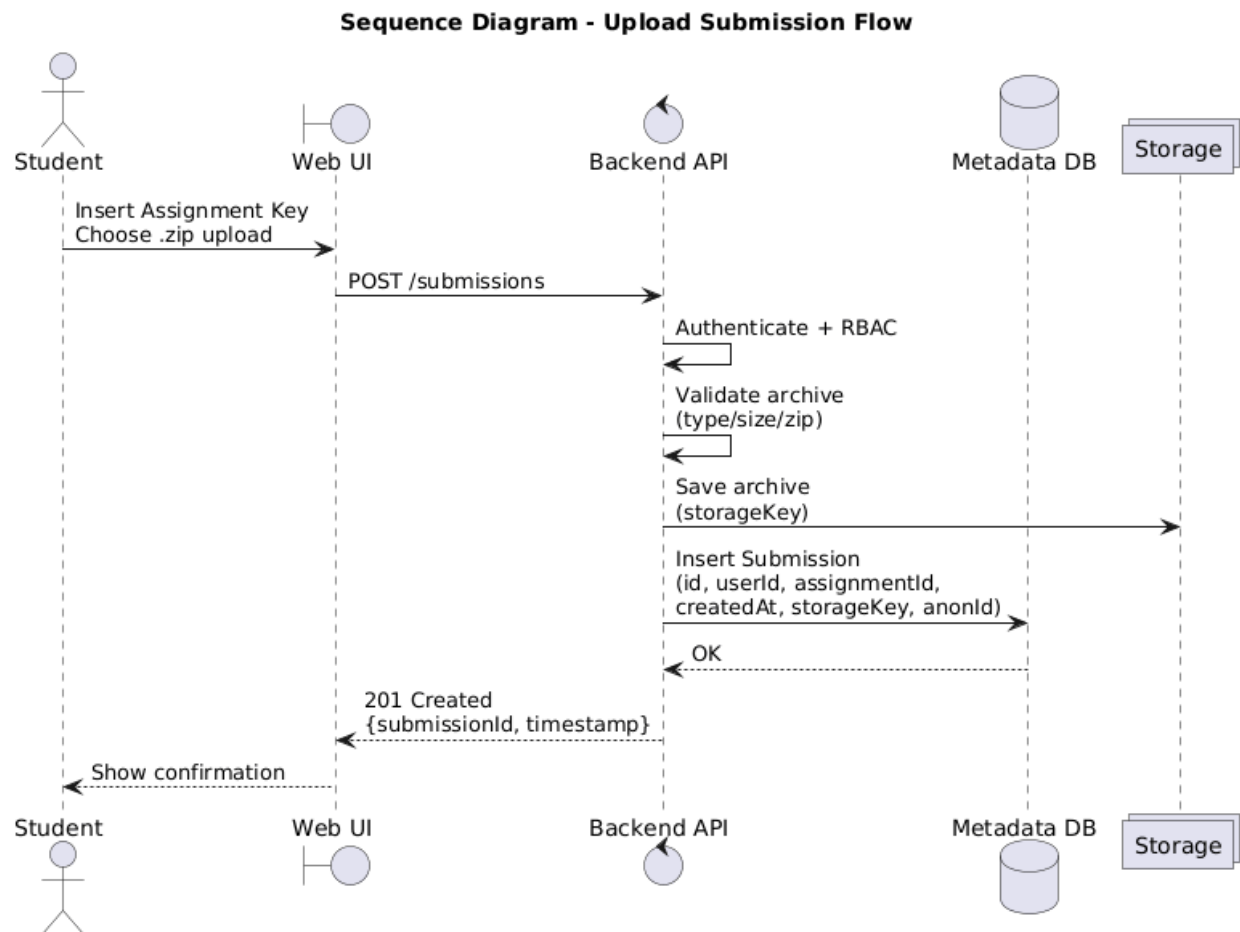


Figure 4: Sequence Diagram: Upload Submission Flow

3.4 Template Management and Mandatory Template Subtraction

3.4.1 Description

Priority: High. Allows professors to upload starter code templates, keeps template versions, marks one version active, and ensures template content is excluded from final similarity scoring.

3.4.2 Stimulus/Response Sequences

- Professor uploads a template archive → system stores a new template version and prepares template artifacts.
- Professor uploads another template → system creates a higher version number and records active status.
- Comparison runs → system uses the active template version, if present, during engine execution.

3.4.3 Functional Requirements

1. **FR-020** The system shall allow professors to upload template archives for an assignment.
2. **FR-021** The system shall version templates and retain previous versions.
3. **FR-022** The system shall mark at most one template version as active for an assignment at a time.
4. **FR-023** The system shall prepare and store the template representation needed by the comparison engine.
5. **FR-024** The system shall subtract template-derived matches before final similarity scoring.
6. **FR-025** The system shall record which template version was used for each comparison run when a template is active.

3.5 Historical Submission Upload for Cross-Term Comparison

3.5.1 Description

Priority: Medium. Supports professor-managed historical archives that are attached to the assignment and used as comparison baselines.

3.5.2 Stimulus/Response Sequences

- Professor uploads a historical archive → system stores the archive, extracts child submissions, and marks them as historical.
- Professor runs a comparison → system includes current-vs-current and current-vs-historical pairs.

3.5.3 Functional Requirements

1. **FR-026** The system shall allow professors to upload historical submission archives for an assignment.
2. **FR-027** The system shall represent historical uploads as historical submissions associated with that assignment.
3. **FR-028** The system shall compare current submissions against other current submissions and against historical submissions for the same assignment.
4. **FR-029** The system shall not require redeployment to add or remove historical comparison inputs.

3.6 Similarity Analysis Jobs

3.6.1 Description

Priority: High. Similarity analysis runs asynchronously in the background and stores scores, match counts, and code/comment evidence spans.

3.6.2 Stimulus/Response Sequences

- Professor triggers analysis → system validates readiness → queues a comparison run → worker processes the run → results are stored.
- Professor checks assignment status → system returns readiness/currentness state and latest comparison results.

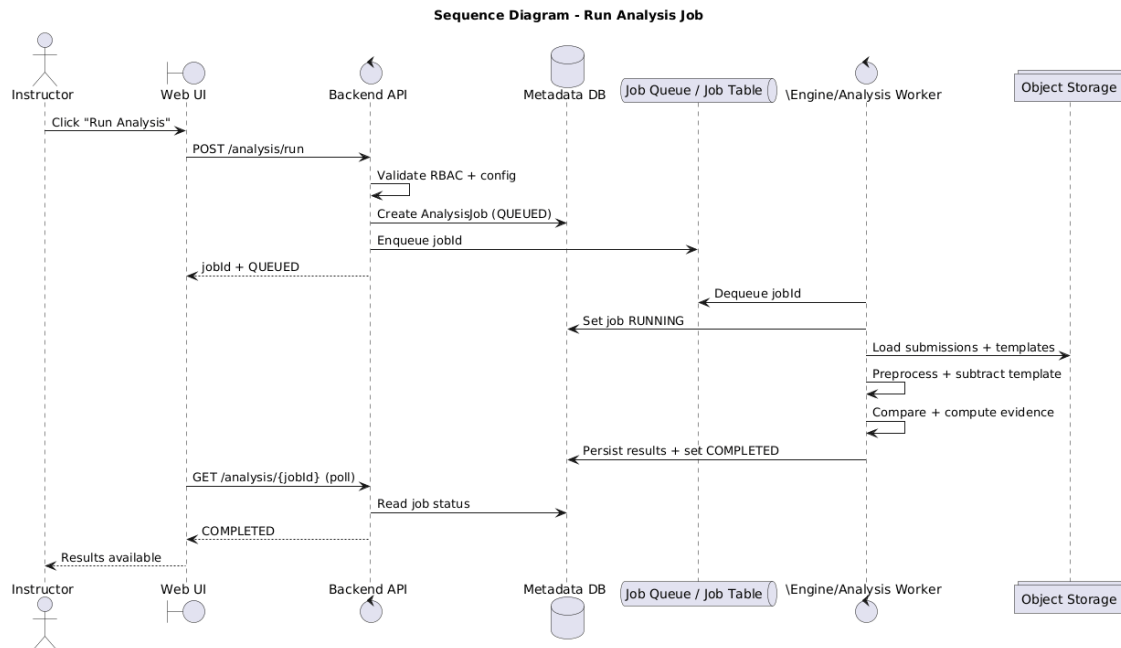


Figure 5: Sequence Diagram: Run Similarity Analysis Job

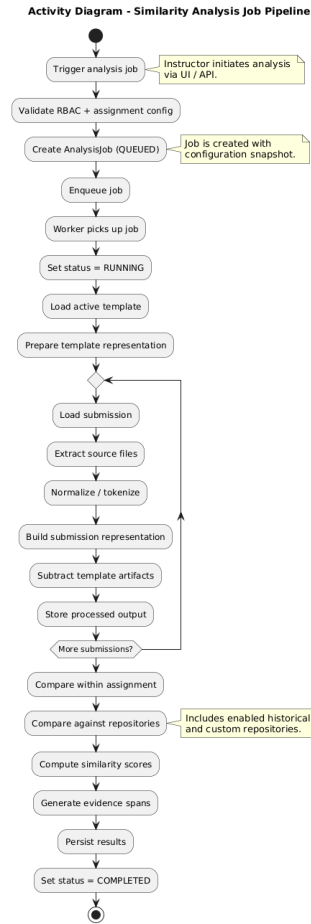


Figure 6: Activity Diagram: Similarity Analysis Pipeline

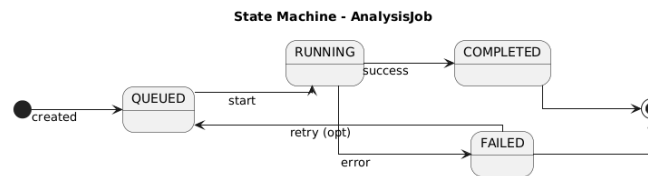


Figure 7: State Machine Diagram: AnalysisJob Lifecycle

3.6.3 Functional Requirements

1. **FR-030** The system shall allow professors to trigger a comparison run for an assignment.
2. **FR-031** The system shall execute upload preparation and comparison analysis asynchronously in a background worker.
3. **FR-032** The system shall require at least one current submission and at least two non-template submissions in total before a comparison run can start.

4. **FR-033** The system shall compute similarity scores for current-vs-current and current-vs-historical submission pairs.
5. **FR-034** The system shall store comparison run status as QUEUED, RUNNING, COMPLETED, or FAILED.
6. **FR-035** The system shall store engine version, parameter snapshot, timestamps, and results for each comparison run.
7. **FR-036** The system shall track whether the latest completed comparison run is current, stale, failed, preparing, running, ready, or not ready at the assignment level.

3.7 Professor Review and Pair Comparison View

3.7.1 Description

Priority: High. Provides a professor-facing review workflow for suspicious pairs, evidence highlighting, identity reveal, and archive download.

3.7.2 Stimulus/Response Sequences

- Professor opens an assignment workspace → system shows suspicious pairs grouped by pair category.
- Professor opens a pair → system shows side-by-side source, matched regions, similarity metrics, and comment evidence counts.
- Professor requests identity reveal → system returns the authorized identity form for that submission when available.

3.7.3 Functional Requirements

1. **FR-037** The system shall display suspicious pairs for completed comparison runs.
2. **FR-038** The system shall separate pair review into current-vs-current and current-vs-historical views.
3. **FR-039** The pair comparison view shall show side-by-side source code with highlighted matched regions when evidence is available.
4. **FR-040** The system shall display matched-token counts and comment-match counts for each pair.
5. **FR-041** The system shall allow professors to reveal submission identity only through authorized professor workflows.
6. **FR-042** The system shall allow professors to download individual submission archives, template archives, and all current submissions for an assignment.

3.8 Privacy and Identity Protection

3.8.1 Description

Priority: High. Protects identity data in the public submission path and restricts access to identifying information.

3.8.2 Stimulus/Response Sequences

- Public submitter enters identifying fields → browser encrypts identity using the public key before upload.
- Professor reviews suspicious pairs → system shows anonymized display labels by default and reveals identity only on demand where supported.

3.8.3 Functional Requirements

1. **FR-043** The public submission flow shall encrypt student identity information before it is sent to the backend.
2. **FR-044** The system shall store encrypted identity for normal public student submissions.
3. **FR-045** The professor review interface shall default to non-raw identity presentation.
4. **FR-046** The system shall restrict identity revelation to the professor who manages the assignment.
5. **FR-047** The public upload-status flow shall expose only the status of the specific upload referenced by its status token.

4 Data Requirements

4.1 Logical Data Model

Core entities in the implemented system include:

- **User**: professor identity and session-backed authentication data.
- **Assignment**: the primary unit of organization for submissions, templates, keys, and comparison runs.
- **AssignmentKey**: public key-like access string used by submitters on the public upload page.
- **UploadBatch**: one ingest event for a current, historical, or template archive.
- **Submission**: one prepared current or historical submission associated with an assignment.
- **AssignmentTemplate**: one prepared template version for an assignment.
- **ComparisonRun**: one async comparison execution snapshot.
- **PairResult**: one pairwise similarity result.
- **PairMatch**: one stored evidence span group within a pair result.
- **Session**: one professor login session.

4.2 Data Dictionary

Entity	Key Fields	Description	Notes
User	id, email, passwordHash, role, createdAt	Authenticated application user.	Implemented professor-facing auth uses this model.
Assignment	id, title, language, dueDate, professorId, comparisonInputVersion	Primary container for keys, uploads, submissions, templates, and comparison runs.	No course/offering entity exists.
AssignmentKey	id, publicKey, assignmentId, isActive	Public submission key bound to an assignment.	Used by student/TA public upload flows.
UploadBatch	id, assignmentId, uploaderId, encryptedIdentity, purpose, status, originalObjectKey	Tracks one uploaded archive and its processing state.	Purpose is current student submission, historical upload, or template upload.
Submission	id, assignmentId, uploadBatchId, displayName, kind, concatenatedSource, sourceMap.Json	Prepared current or historical submission.	Kind is CURRENT or HISTORICAL.
AssignTemplate	id, assignmentId, uploadBatchId, versionNumber, isActive, concatenatedSource, sourceMap.Json	Prepared template version for an assignment.	Used for mandatory template subtraction.
ComparisonRun	id, assignmentId, templateVersionId, inputVersion, status, engineVersion, paramsJson	One asynchronous comparison execution.	Stores engine and parameter snapshot.
PairResult	id, comparisonRunId, leftSubmissionId, rightSubmissionId, similarityScore, commentScore, matchedTokenCount	Pairwise similarity result.	Used by suspicious-pair UI.
PairMatch	id, pairResultId, matchKey, kind, byte/token/line spans	Stored evidence region for one pair.	Kind is CODE or COMMENT.
Session	id, tokenHash, userId, expiresAt, lastUsedAt	Session record for professor login.	Cookie-backed authentication.

4.3 Data Acquisition, Integrity, Retention, and Disposal

This section describes how the implemented system acquires data, preserves integrity, and applies retention and deletion behavior.

- **Data acquisition.** The system acquires current submissions from the public assignment-key upload flow, bulk current submissions from the TA/marker bulk upload flow, and historical/template archives from authenticated professor uploads.
- **Integrity and validation.** The system shall validate uploads to prevent unsafe content, including unsupported formats, unsafe archive paths, excessive nesting, excessive file counts, and oversized extracted archives. Template versions shall be retained as versioned records. Comparison results shall reference the input version and template version used to produce them.
- **Retention.** The system shall retain submissions, templates, upload batches, comparison results, and related artifacts for at least the duration of the assignment, subject to deployment policy and authorized deletion operations.
- **Disposal and deletion.** When an authorized professor deletes an assignment, submission, template, or category of artifacts, the system shall delete the related metadata and best-effort remove associated stored object files. Comparison data shall be cleared when required by those deletion operations.
- **Privacy constraints.** Public submitters do not receive account-based access to stored submissions. Public status access shall be limited to the specific upload referenced by its status token. Professor identity reveal shall remain protected by authenticated ownership checks.

5 External Interface Requirements

5.1 User Interfaces

- **Public Submission Page:** single student zip upload, required identity fields, assignment key entry, upload status feedback, and help documentation.
- **Bulk Upload Section:** TA/marker convenience flow on the same public page for one parent zip containing child submission zips.
- **Professor Login and Signup:** professor-only authentication pages.
- **Professor Dashboard:** create assignments, view due dates, copy assignment keys, and monitor submission/comparison state.
- **Assignment Workspace:** upload historical archives and templates, run comparisons, inspect upload batches, manage deletions, and download artifacts.
- **Pair Comparison View:** side-by-side source display, match highlighting, counts, and identity reveal tools.

5.2 Software Interfaces

- **Database:** PostgreSQL storing users, assignments, keys, upload batches, submissions, templates, comparison runs, pair results, matches, and sessions.
- **Queue:** Redis-backed BullMQ queues for upload preparation and comparison runs.
- **Storage:** filesystem-backed object storage for raw uploaded archives and prepared files.
- **Engine Interface:** worker-to-engine communication via child-process execution and JSON over stdin/stdout.
- **Authentication:** cookie-backed professor sessions; assignment-key and status-token based public submission/status access.

5.3 Hardware Interfaces

The system is a web-based application and does not require specialized hardware interfaces beyond standard client and server hardware.

- **Client hardware.** Users access the system from a desktop or laptop computer capable of running a modern web browser with JavaScript enabled.
- **Input devices.** The system supports standard keyboard and mouse/trackpad input for login, navigation, upload, and review workflows.
- **Display.** The pair viewer shall support side-by-side display on typical desktop or laptop screens.
- **File upload capability.** Client devices shall support selecting and uploading zip files from local storage. The implemented public flow expects zip archives rather than single loose source files.
- **Server hardware.** The server host shall provide enough CPU, memory, and storage to run the API, worker, database, Redis, and artifact storage; the worker and engine are the most CPU-sensitive components.

5.4 Communications Interfaces

- The system shall use HTTPS for client-server communications in non-local deployments. **(NFR-003)**
- The system shall support multipart HTTP uploads for archive submission.
- The API and worker shall communicate asynchronously through Redis-backed queues.
- The worker shall communicate with the engine through local process execution and standard I/O, not through a network API.

6 Quality Attributes

6.1 Usability

- **NFR-005** The public submission flow shall make required fields explicit and provide clear success/failure feedback after upload.
- **NFR-006** The system shall present upload processing state to public submitters after submission.
- **NFR-007** The professor review interface shall clearly distinguish current-vs-current pairs from current-vs-historical pairs.
- **NFR-008** The comparison view shall provide clear highlighting and navigation between matching regions.
- **NFR-009** Error messages shown to end users shall be actionable and shall not expose sensitive internal details.

6.2 Performance

- **NFR-010** Upload acceptance should return promptly and defer heavy processing to background jobs.
- **NFR-011** For a class-sized assignment, the system should complete same-assignment comparison processing within an operationally acceptable time on baseline deployment hardware.
- **NFR-012** Triggering a comparison run should enqueue work quickly and return control to the professor without blocking on the full analysis.
- **NFR-013** The system should avoid unnecessary duplication of prepared data across repeated comparison runs by reusing prepared submission and template artifacts until inputs change.

6.3 Security

- **NFR-014** The system shall enforce deny-by-default authorization on protected professor endpoints.
- **NFR-015** The system shall validate uploads to prevent malicious archive structures and unsafe extraction paths.
- **NFR-016** The system shall protect secrets and cryptographic key material through configuration rather than hard-coding.
- **NFR-017** The system shall encrypt student identity data in the public submission flow before transmission.
- **NFR-018** The system shall prevent public submitters from accessing stored submissions, pair results, or other users' upload data; only the status of the specific submitted upload may be exposed through its status token.

7 Use Cases

7.1 UC-01: Public Student Submits an Assignment

Primary Actor(s): Student Submitter **Preconditions:** A valid assignment key has been provided by the professor or course staff; the public submission page is available. **Trigger:** The student enters submission details and uploads a zip archive using the assignment key.

7.1.1 Main Success Scenario

1. The student opens the public submission page.
2. The student enters name, student number, optional email, and assignment key.
3. The student selects one zip archive.
4. The browser encrypts identity information using the public submission identity key.
5. The system validates the archive and stores it as a new upload batch.
6. The system queues background preparation for that upload.
7. The system returns upload-tracking information and displays submission status in the UI.
8. The student remains on the page and sees whether the upload becomes ready or fails.

7.1.2 Alternate / Exception Flows

- Wrong or inactive assignment key → system rejects the upload.
- Uploaded file is not a zip archive → system rejects the upload.
- Archive contains no supported source files or invalid structure → system marks the batch failed and returns a user-facing error.

Postconditions: The upload batch is recorded, preparation is attempted asynchronously, and the student can view status for that upload only. The student does not receive an account-based submission history.

7.2 UC-02: Professor Uploads Template or Historical Archive

Primary Actor(s): Professor **Preconditions:** Professor is authenticated; assignment exists and is managed by that professor. **Trigger:** Professor uploads a template archive or a historical archive in the assignment workspace.

7.2.1 Main Success Scenario

1. The professor opens an assignment workspace.
2. The professor selects either a template upload or a historical upload.
3. The professor uploads a zip archive.
4. The system validates the archive and stores it as an upload batch.

Use Case Diagram - Student

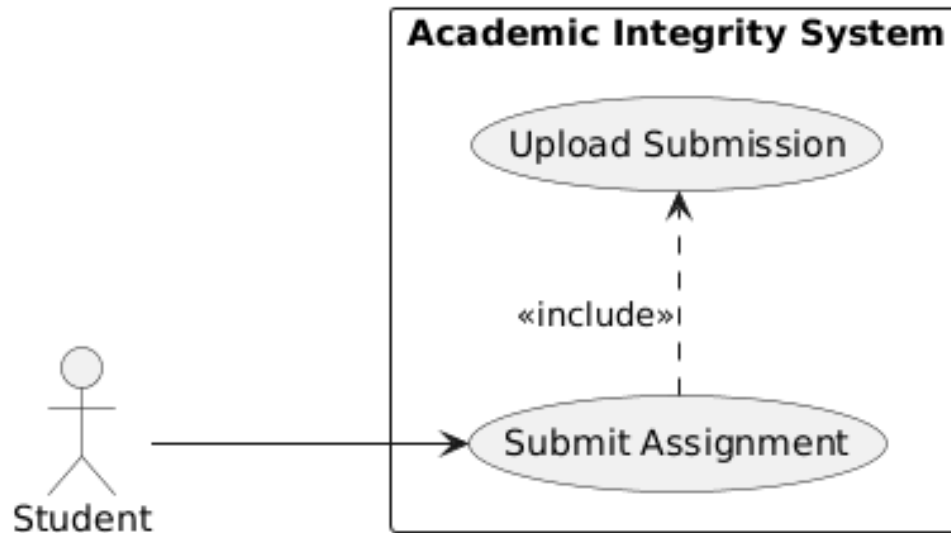


Figure 8: Use Case Diagram: Student

5. The worker prepares the archive into template artifacts or historical submissions.
6. The workspace refreshes and shows the new template version or historical submissions.

7.2.2 Alternate / Exception Flows

- Historical archive has invalid child-zip layout → system rejects the upload or returns warnings.
- Template archive is invalid or unsupported → system marks the upload failed and reports the reason.
- Assignment is not owned by the professor → system denies access.

Postconditions: A new template version or historical submission set is attached to the assignment. Historical uploads become available for current-vs-historical comparison in later runs.

7.3 UC-03: Professor Runs Similarity Analysis

Primary Actor(s): Professor **Preconditions:** Professor is authenticated; the assignment has at least one current submission and at least two non-template submissions in total; no uploads or comparison runs are currently active. **Trigger:** Professor clicks *Run Comparison*.

7.3.1 Main Success Scenario

1. The system validates that the assignment is ready for comparison.
2. The system creates a ComparisonRun record containing the input version, engine version, active template version if any, and configured parameters.

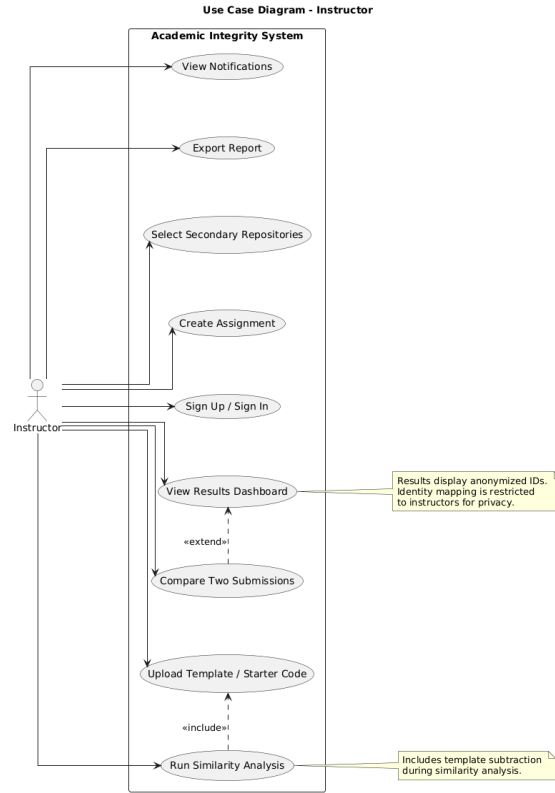


Figure 9: Use Case Diagram: Instructor

3. The system queues the comparison run and returns immediately.
4. The worker loads prepared current submissions, historical submissions, and the active template if available.
5. The worker generates allowed pairs for current-vs-current and current-vs-historical comparison.
6. The worker launches the engine and passes the analysis request over stdin.
7. The engine returns pair results and evidence spans.
8. The worker stores results and marks the run completed.

7.3.2 Alternate / Exception Flows

- Uploads are still being prepared → system blocks comparison start.
- Not enough submissions are available → system does not create the run.
- Engine or worker failure occurs → system marks the run failed and stores an error message.

Postconditions: Comparison results are stored and the assignment workspace reflects current, stale, failed, or completed state as appropriate.

7.4 UC-04: Professor Reviews Suspicious Pairs

Primary Actor(s): Professor **Preconditions:** At least one comparison run has completed.

Trigger: Professor opens the suspicious-pairs section or a specific pair view.

7.4.1 Main Success Scenario

1. The system displays pair groups for current-vs-current and current-vs-historical results.
2. The professor selects a suspicious pair.
3. The system loads both prepared submissions and stored match regions.
4. The system renders a side-by-side comparison view with highlighted evidence.
5. The professor reviews similarity score, matched-token count, and comment-match count.
6. The professor optionally reveals identity for one or both submissions when that information is available.

7.4.2 Alternate / Exception Flows

- A pair has low code similarity but supporting comment evidence → system still presents it for manual review.
- Identity information is unavailable in raw form for a submission → system presents only the supported reveal mode or no reveal option.
- Professor attempts to access a pair outside their assignment ownership → system denies access.

Postconditions: The professor can inspect code evidence and identity information, when authorized, to support academic-integrity review.

8 UML and Design Documentation

8.1 Key Architectural Decisions

- **ADR-001 Assignment-centric data model.** The implemented application uses assignments as the primary container for keys, uploads, templates, and comparison runs. Course/offering abstractions were removed from the delivered system.
- **ADR-002 Asynchronous processing.** Upload preparation and similarity comparison execute in background workers to avoid blocking browser requests.
- **ADR-003 Storage separation.** Raw and prepared artifacts are stored in filesystem-backed object storage, while metadata and results are stored in PostgreSQL.
- **ADR-004 Engine isolation.** The comparison engine is delivered as a separate Rust CLI executed by the worker through standard I/O.
- **ADR-005 Template-aware scoring.** Template subtraction is part of the implemented analysis flow and is applied before final similarity scoring.

8.2 Diagram Update Notes for Version 2.0

All UML and architecture diagrams maintained alongside this SRS shall be updated to reflect the implemented system. In particular:

- remove all course/offering entities and relationships,
- remove repository-selection diagrams that imply multiple secondary repository objects,
- show assignment-key based public submission instead of student login,
- include the TA/marker bulk current-upload option,
- show current-vs-current and current-vs-historical comparison only,
- and show the worker-to-engine interaction as a local CLI process invocation rather than a service call.

9 Glossary

Term	Definition
Tokenization	Breaking source code into tokens (keywords, identifiers, operators, literals).
Normalization	Transforming tokens to reduce superficial differences (e.g., renaming identifiers to ID).
Fingerprint	A compact set of hashed features representing a submission after pre-processing.
Winnowing	A fingerprint-selection method that chooses representative hashes from sliding windows.
GST (Greedy String Tiling)	Algorithm that finds non-overlapping matching token substrings (tiles) to provide evidence.
Template Subtraction	Removing template-derived features from submissions before computing similarity scoring.
Repository	A collection of submissions used as a comparison source (current offering or secondary).
Offering	A specific course instance/term (e.g., Winter 2026).
RBAC	Role-Based Access Control: enforcing permissions based on user role.